Secondary networks for KubeVirt VMs with DRA



Multus and DRA on kube-ovn, side by side

Sören Henning · Kubermatic

KubeVirt Summit 2026

Why look beyond Multus?

KUBERMATIC

1

Scheduler is blind

Secondary-NIC assignment happens at CNI ADD, after scheduling. The scheduler cannot refuse or place on it.

2

No lifecycle object

Nothing declares "this pod owns this IP". A NAD is configuration, not an allocation.

3

Serial attach

Multus calls delegate CNIs one after another, so latency grows with the number of NICs.

The question

Does DRA do better for kube-ovn's SDN NICs?

From KNDM to kube-ovn

KUBERMATIC

KNDM
Kubernetes Network Driver Model: declarative networking on DRA + NRI, instead of CNI and Multus chains

→

DraNet
The implementation (kubernetes-sigs): discovers host devices and moves them into the pod

→

This work
Same DRA + NRI pattern, with an SDN backend (kube-ovn) instead of hardware

arXiv 2506.23628

Hardware backend (DraNet)
e.g. PCI NICs, SR-IOV VFs

- Discovers real devices on the host
- Attach = move an existing device into the netns

SDN backend (kube-ovn)
nothing to discover

- The NIC is synthesized on demand
- Attach = create: veth, OVS port, IP, logical port

Design principle

The controller owns allocation. The node side only plugs.

kube-ovn in one slide

KUBERMATIC

OVN
SDN control plane. Logical switches and routers become OVS flows on every node.

VPC
Isolated L3 domain with its own logical router.

Subnet
IP pool + L2 domain, always in exactly one VPC.

Overlay
OVN virtual network between nodes

Underlay
real VLAN via a ProviderNetwork

Flat IPAM

- One CIDR per subnet, across all nodes
- The logical switch spans every node
- The scheduler picks the node, then the IP comes from the pool

Not a /27 per node like the default pod CIDR

kube-ovn-controller

- Resolves device → subnet → provider
- IPAM: IP + MAC, logical port, IP CR
- Writes the pod annotations
- GC keeps a stopped VM's ports

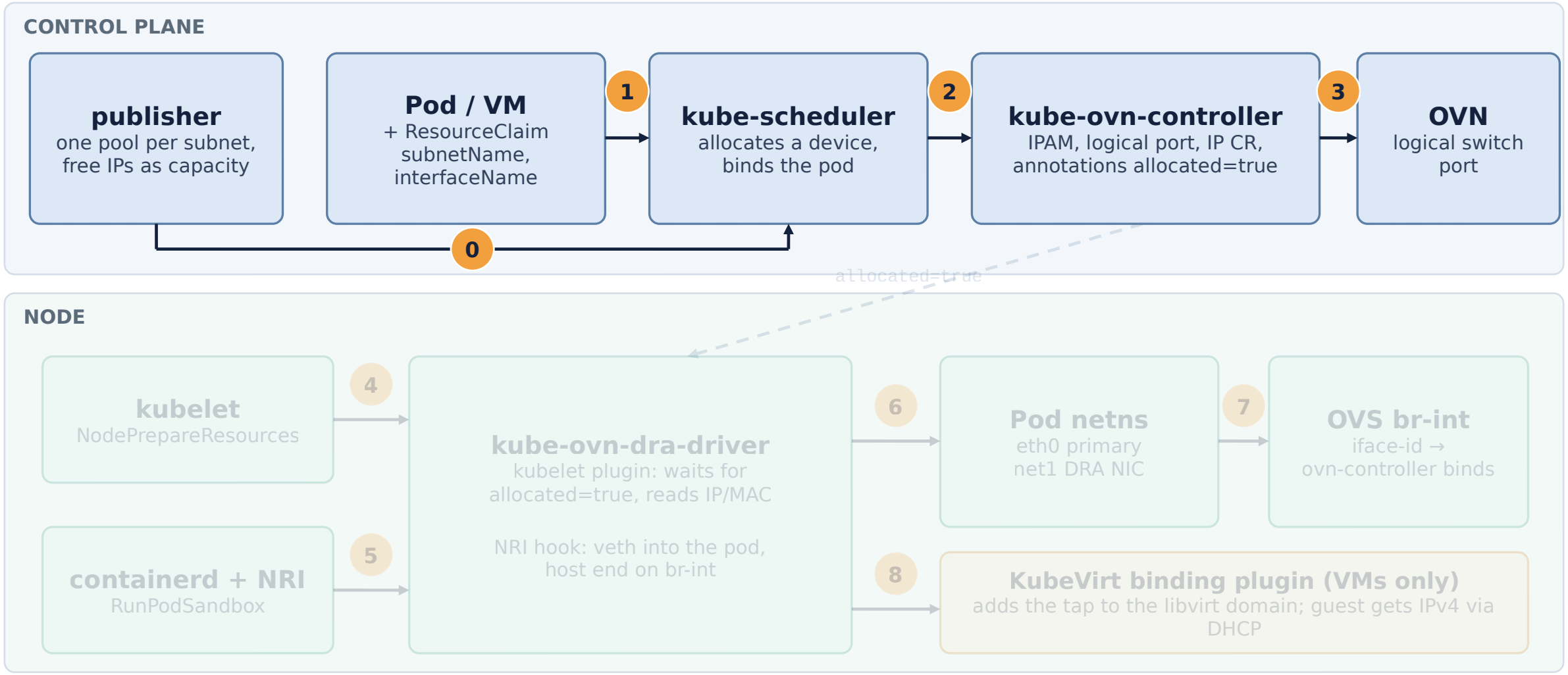
kube-ovn-dra-driver

- Publisher: one pool per subnet, free IPs as capacity
- Plugin: waits for allocated=true
- Plugin: plugs the NIC (VMs: tap + DHCP)
- Plugin: detaches on pod delete

Control plane: the claim decides

KUBERMATIC

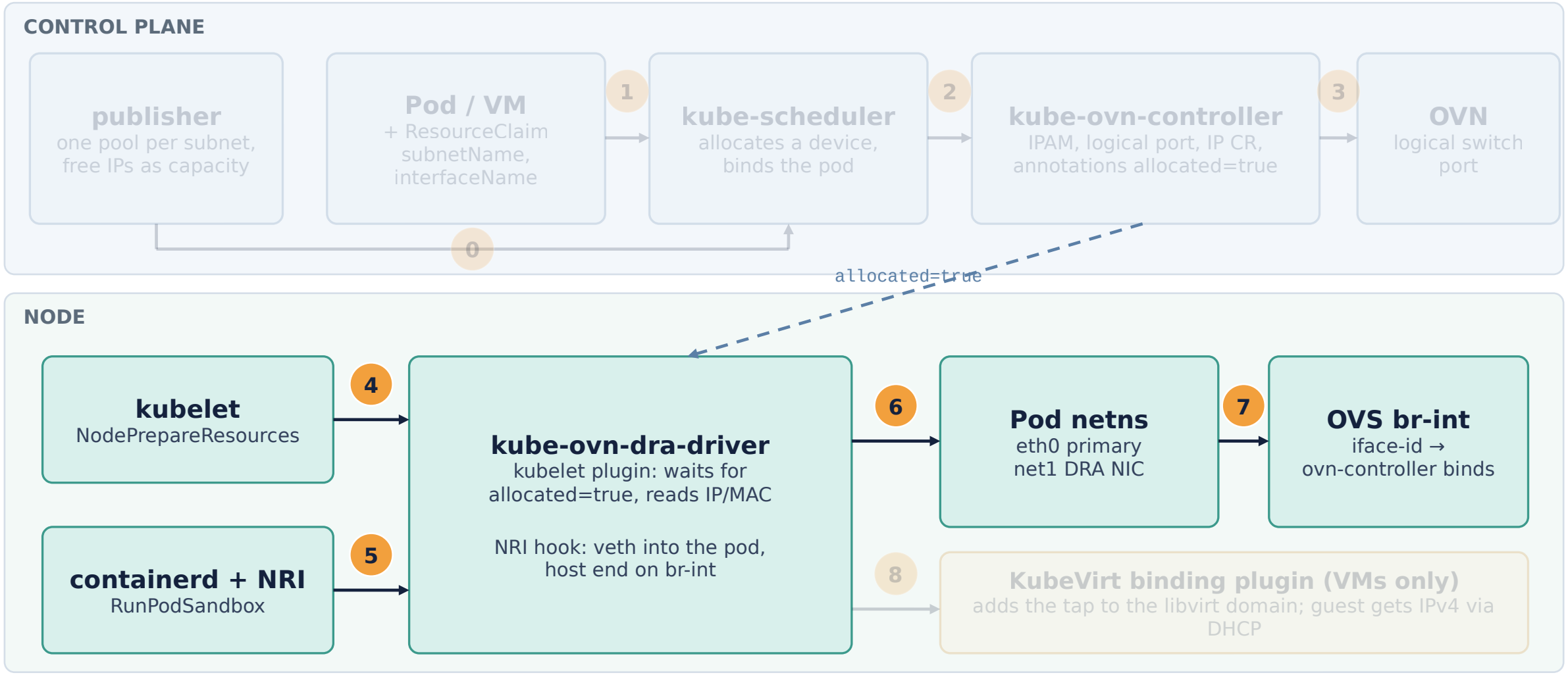
The publisher announces the subnets and their free IPs. The claim is a scheduling input; kube-ovn allocates.



Node: the driver only plugs

KUBERMATIC

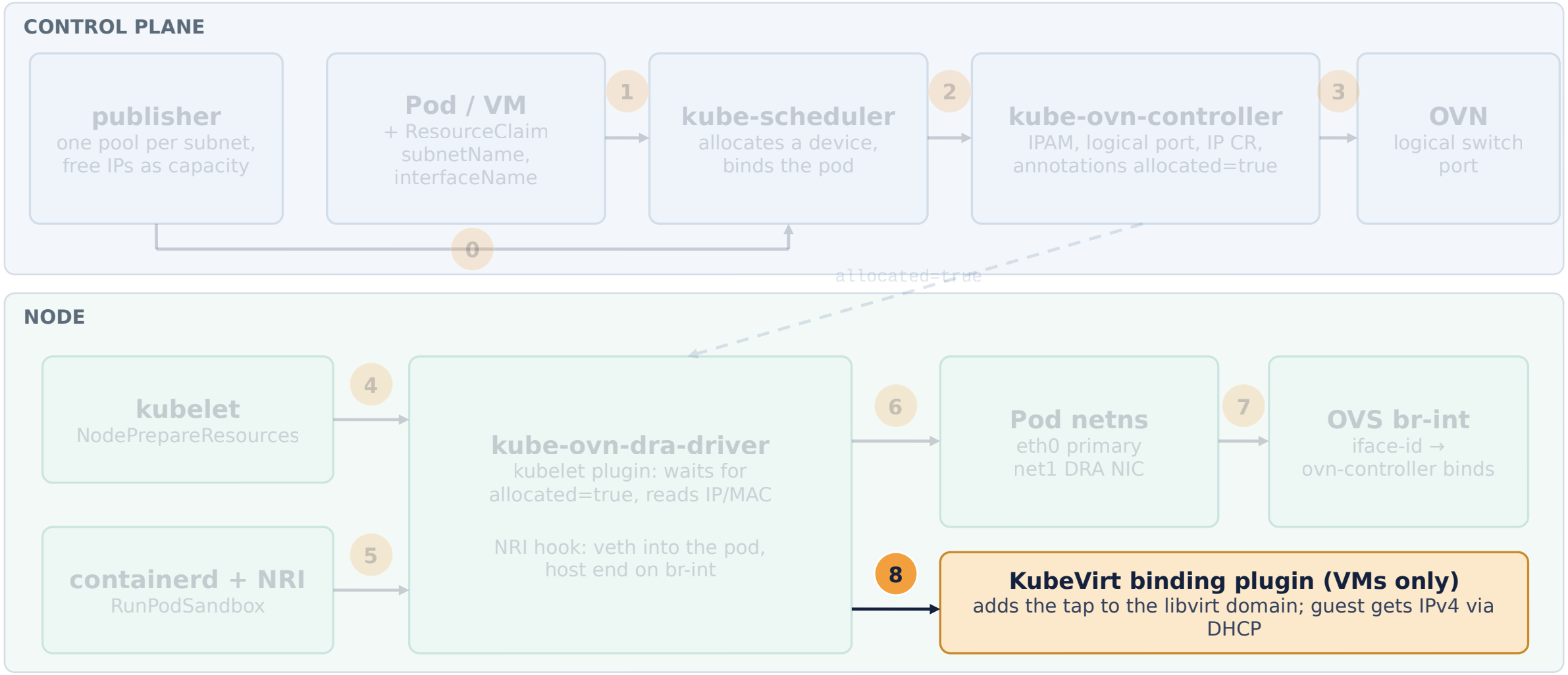
At Prepare the pod netns does not exist yet, and the NRI window is about 2 s. Hence two phases.



VMs: from tap to domain

KUBERMATIC

For VMs the driver also builds the bridge, the tap and a small DHCP server. The sidecar adds the tap to the VM.



Subnets become devices

KUBERMATIC

kube-ovn Subnet

```
kind: Subnet
metadata:
  name: ovn-subnet
spec:
  cidrBlock: 10.200.0.0/24
  vpc: ovn-cluster
```

published as →

ResourceSlice device (pool: ovn-subnet)

```
name: subnet-ovn-subnet
attributes:
  subnetName: ovn-subnet
  subnetType: ovn
  provider: ovn-subnet.default.ovn
capacity:
  nic.kubeovn.io/ipv4: 250
```

One pool per subnet. Overlay: all nodes. VLAN: only the nodes where its provider network is ready.

A claim selects it with CEL

```
device.attributes['nic.kubeovn.io']
  .subnetName.stringValue == 'ovn-subnet'
```

A subnet is a shared IP pool, but DRA allocates exclusively by default. The 2nd pod stays Pending until the device sets `AllowMultipleAllocations: true` (DRAConsumableCapacity: alpha in 1.35, beta from 1.36). The free addresses are also published as capacity, so a full pool is refused at scheduling time.



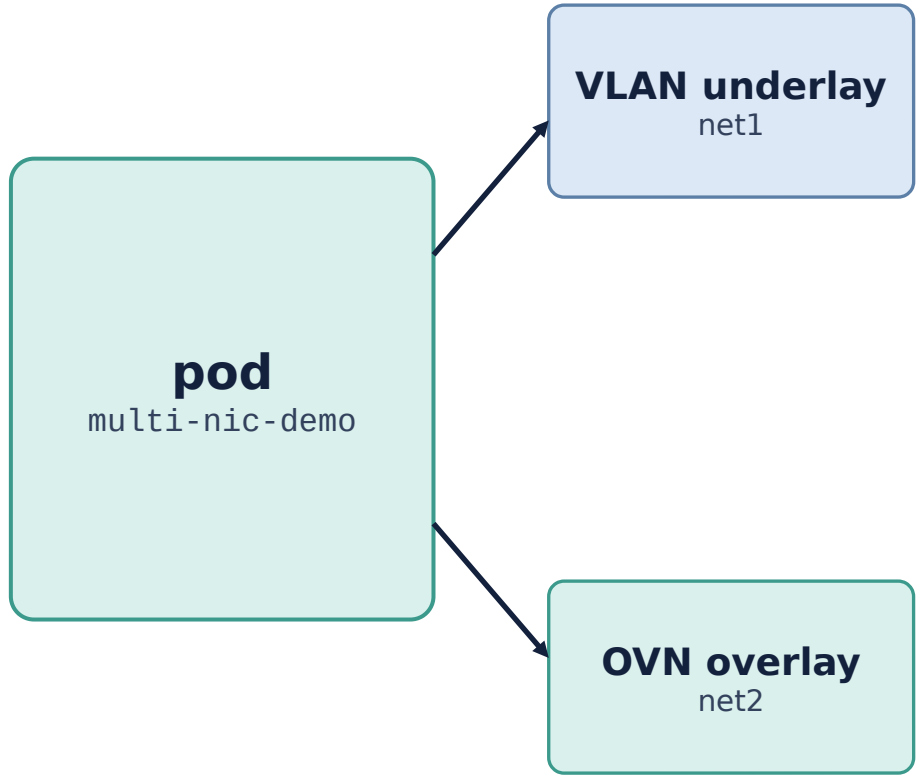
Demo: two NICs, one pod

KUBERMATIC

1. **`make kind-demo`**
   or a pre-staged cluster

2. **`kubectl get resourceslices`**
   the published devices, one per subnet

3. **`kubectl exec multi-nic-demo -- ip addr`**
   net1 (VLAN underlay) and net2 (OVN overlay)



No hot-plug: NICs are attached when the sandbox starts.

Multus vs DRA at a glance

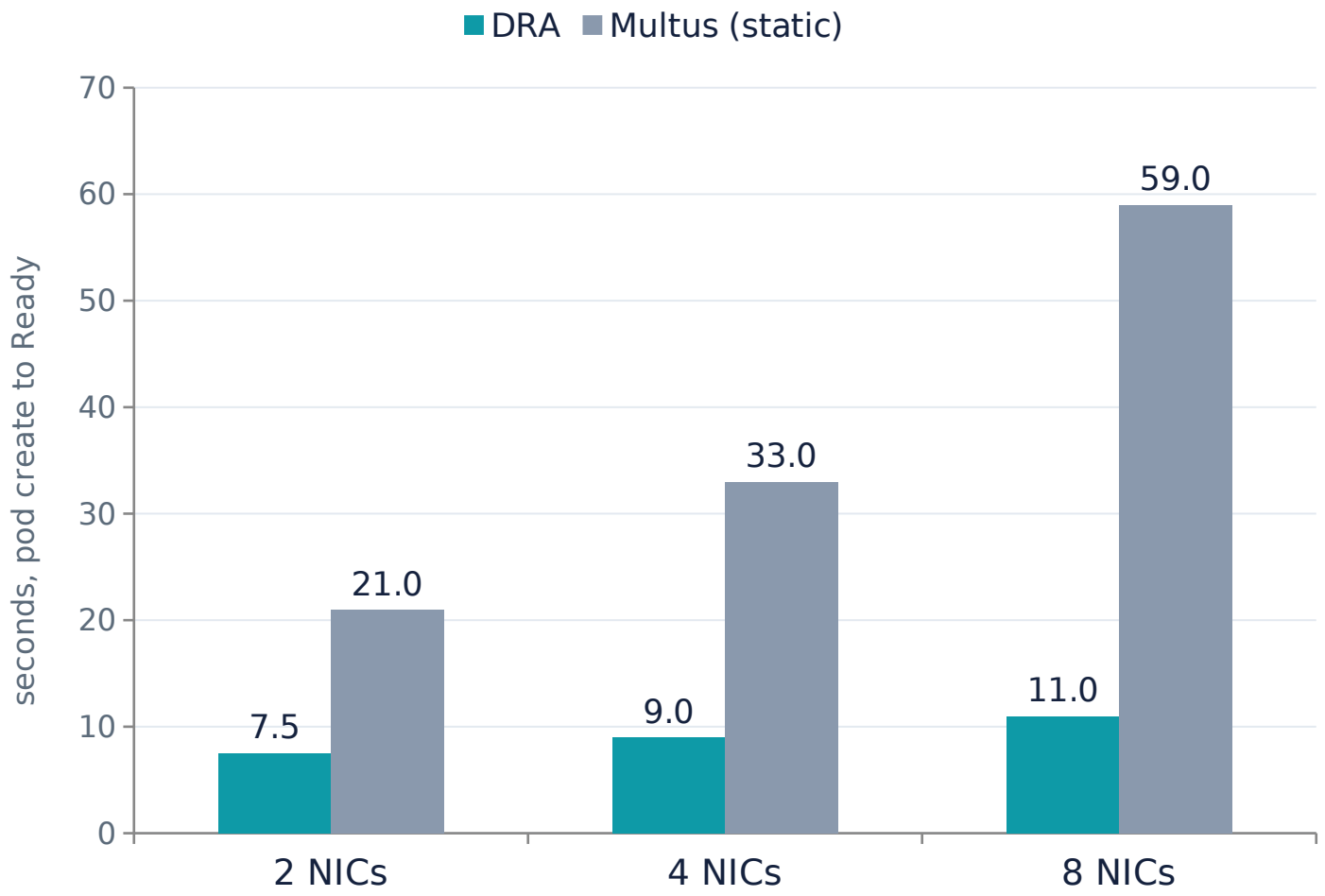
KUBERMATIC

Aspect	Multus + NAD	DRA claim
Scheduler visibility	Annotation, invisible to the scheduler.	Claim is a scheduling input. A full pool is refused at scheduling time.
Shared subnet	No concept of exclusivity.	Exclusive by default; AllowMultipleAllocations fixes it.
Lifecycle	A NAD is configuration.	A claim has allocate and release states.
Hot-plug	KubeVirt hot-plugs Multus NICs its own way.	Claims are immutable; VEP-183 non-goal.
Operations	Separate NAD and CNI chain.	One allocation path in kube-ovn-controller.

For a plain node-agnostic overlay there is no difference. For underlay, the pool of a VLAN subnet selects the nodes of its provider network.

Attach time: DRA stays flat

KUBERMATIC



7–12 s

DRA, 2 to 8 NICs (all 90 runs)

21 → 59 s

Multus static, +13 s per extra pair of NICs

~21 s

hot-plug: Ready with 2 NICs; 8 NICs in total ~54 s

Overlay mode, median of 10 runs; underlay and mixed within about 1 s. kind 1.35, 1 control-plane + 1 worker, homelab node.

Takeaways

KUBERMATIC

1 **Complementary, not a verdict**
Multus and DRA differ in visibility, IP handling and lifecycle.

2 **On an SDN, the work is in the control plane**
The node side is thin.

3 **So it belongs close to the SDN's control plane**
Allocation in the controller, only plugging on the node.

```
github.com/soer3n/kube-ovn-dra-driver
github.com/soer3n/kube-ovn-network-binding-plugin
```
Questions, feedback: the kube-ovn repository, SIG-Network, KubeVirt

Backup: where each piece fits

KUBERMATIC

DraNet, SR-IOV DRA
Hardware passthrough. Exclusive or partitionable devices.

ovs-cni
Trunk ports on a pre-existing OVS bridge. Rides on Multus.

Multus + NAD
The established secondary-NIC path. Scheduler-blind.

This driver
Scheduler-visible virtual SDN endpoints for kube-ovn.

Complementary layers. A cluster can run more than one.